

Ashpaz Data Pipeline

Q2: Kafka Consumer Implementation

Parastou Peiravifar, Pouya Anvari, Faezeh Khanmohammadi

This document outlines the architecture, implementation, and execution results for the real-time data ingestion layer of the Ashpaz food delivery platform. It details the Kafka consumer design, stateless validation rules, and fundamental architectural decisions regarding high-throughput streaming.

May 18, 2026

Contents

1	Introduction	2
2	Questions	2
2.1	Kafka vs. Relational Databases	2
2.2	Stateless vs. Stateful Processing	2
2.3	Kafka as a Distributed Commit Log	3
3	Implementation Details: The Validation Consumer	4
3.1	Validation Rules Engine	4
3.2	Routing Protocol	4
4	Execution Outputs and System Logs	5
4.1	Terminal Output	5
4.2	Data Payload: Valid Event Structure	5
4.3	Data Payload: Error Event Structure	5
5	Conclusion	6

1 Introduction

The Ashpaz platform requires a robust, high-throughput data pipeline to process real-time restaurant orders. As part of this data science initiative, the system has transitioned from standard relational data designs to a live streaming architecture. This report focuses specifically on the **Ingestion and Validation Layer**, utilizing Apache Kafka to consume, validate, and route incoming transaction events dynamically.

2 Questions

This section addresses the theoretical foundation of the ingestion layer and justifies the technological stack selected for the pipeline.

2.1 Kafka vs. Relational Databases

Apache Kafka was explicitly chosen as the ingestion layer over traditional relational databases (RDBMS) such as PostgreSQL or MySQL due to its superior capabilities in handling real-time streaming data.

- **Performance and Scalability:** Kafka operates as an append-only distributed log, optimized for sequential disk writes. This allows it to absorb massive bursts of high-velocity order events (millions of messages per second) without the locking and indexing overhead inherent to RDBMS.
- **Data Access Patterns:** While SQL databases are optimized for complex queries and stateful joins, Kafka is designed for sequential reads and real-time processing pipelines.
- **When an RDBMS is preferred:** A relational database remains the superior choice for the final storage of structured data where complex aggregations (e.g., monthly financial reporting), ACID compliance, and ad-hoc analytical queries are required.

2.2 Stateless vs. Stateful Processing

The deployed Kafka consumer explicitly utilizes *stateless validation*.

- **Stateless Processing:** The evaluation of an event depends *only* on the data contained within the event itself. Our consumer validates phone formats and calculates order prices without needing to query a database. This ensures sub-millisecond latency.
- **Stateful Processing:** Requires retaining memory of past events or querying external databases (e.g., checking if a user ID exists in a Postgres table before validating).
- **Risks of Stateful Consumers:** Introducing stateful lookups at the ingestion layer creates severe bottlenecks, increases latency, and complicates horizontal scaling, as consumer partitions must carefully manage local state or rely on network calls.

2.3 Kafka as a Distributed Commit Log

Unlike traditional message queues (like RabbitMQ) which delete messages once consumed, Kafka functions as a distributed commit log. Messages are immutably written to disk and retained for a configurable period. This architecture enables:

1. **Message Replay:** If a downstream component fails or a new analytical model is deployed, consumers can rewind their offset to re-process historical data.
2. **Multiple Independent Consumers:** Different microservices (e.g., fraud detection, database insertion, and real-time dashboards) can read the exact same data stream at their own pace without interfering with one another.
3. **Fault Tolerance:** Partitions are replicated across multiple brokers, ensuring data integrity even if hardware fails.

3 Implementation Details: The Validation Consumer

The consumer application connects to the KRaft-enabled Kafka cluster, polls the `ashpaz.order` topic, and applies critical business logic before routing the data.

3.1 Validation Rules Engine

The following logic validates incoming payloads strictly in-memory:

```
1 def validate_order(order):
2     errors = []
3
4     # 1. Phone Number Format (+91 or 080)
5     phone = order.get("phone_number", "")
6     if not (phone.startswith("+91") or phone.startswith("080")):
7         errors.append("INVALID_PHONE")
8
9     # 2. Order Mode Conflict
10    if order.get("request_online") and order.get("request_table"):
11        errors.append("ORDER_MODE_CONFLICT")
12
13    # 3. Order Price Correctness
14    expected_price = 0.0
15    for item in order.get("items", []):
16        expected_price += (item.get("unit_price", 0.0) * item.get("
17quantity", 1))
18
19    expected_price = round(expected_price, 2)
20    actual_price = order.get("order_price", 0.0)
21
22    if expected_price != actual_price:
23        errors.append("PRICE_MISMATCH")
24
25    return errors
```

Listing 1: Stateless Validation Logic

3.2 Routing Protocol

Once evaluated, the consumer dynamically routes the message. Valid orders are replicated directly to `ashpaz.valid`. Invalid orders are restructured into a specific JSON schema encapsulating the `error_reason` and the `original_order`, then transmitted to `ashpaz.error_log`.

4 Execution Outputs and System Logs

The following sections document the operational outputs of the consumer handling live data from the Ashpaz producer.

4.1 Terminal Output

```
VALID: Order 570bf559-1c96-4bb7-b2f9-45e113c9c73b routed to ashpaz.valid
VALID: Order c41e0169-5bc1-449c-b84e-c22cc9621c2f routed to ashpaz.valid
VALID: Order 3e4dbc85-505d-452c-9d2a-82a8195cdaf7 routed to ashpaz.valid
VALID: Order 1c709fb3-b0bc-444b-8334-94b335ebc392 routed to ashpaz.valid
VALID: Order db4116d8-ad31-40ed-9b2d-e6233a2eb520 routed to ashpaz.valid
INVALID (INVALID_PHONE): Order 2bcbdda5-13e1-47d3-a837-023da301a634 routed to
ashpaz.error_log
VALID: Order 02cfafcb-11e5-4a17-a8ef-77585129ed86 routed to ashpaz.valid
VALID: Order 27e3cedd-1bec-42cc-b7fe-9c8de9489925 routed to ashpaz.valid
VALID: Order 5424e65f-9347-43d0-9248-ed4ac756a88a routed to ashpaz.valid
```

4.2 Data Payload: Valid Event Structure

The following JSON structure represents a successfully processed transaction committed to the `ashpaz.valid` topic:

```
1 {
2   "order_id": "ff2fb2e7-18d6-4765-b36c-7d4bea1481bc",
3   "user_id": "user_000158",
4   "restaurant_id": 11845,
5   "restaurant_name": "Swathi Greenland",
6   "restaurant_city": "Rajajinagar",
7   "cuisines": ["Andhra", "North Indian", "Chinese"],
8   "phone_number": "08023502055",
9   "request_online": true,
10  "request_table": false,
11  "order_time": "2026-05-23T08:20:12.262911Z",
12  "items": [{"food_item": "Roti", "category": "North Indian", "unit_price": 256.02, "quantity": 4}],
13  "order_price": 1024.08
14 }
```

Listing 2: Sample Output - Valid Order

4.3 Data Payload: Error Event Structure

When an order violates the stateless rules, it is serialized into the schema below and routed to `ashpaz.error_log`:

```
1 {
2   "order_id": "0a82dff5-4b36-4fa2-b12c-e2d4d9bd0d3d",
3   "error_type": "PRICE_MISMATCH",
4   "error_reason": ["PRICE_MISMATCH"],
5   "original_order": {"order_id": "0a82dff5-4b36-4fa2-b12c-e2d4d9bd0d3d",
6     "user_id": "user_000295", "restaurant_id": 11955, "restaurant_name":
7     "Beyond Elements", "restaurant_city": "Whitefield", "cuisines": ["
8     Cafe", "Italian", "Pizza", "North Indian"], "phone_number":
```

```

"08048500777", "request_online": false, "request_table": false, "
order_time": "2026-05-23T08:15:04.857703Z", "items": [{"food_item":
"Unlisted Mystery Dish", "category": "Unknown", "unit_price":
432.17, "quantity": 1}, {"food_item": "Pasta", "category": "Italian
", "unit_price": 337.99, "quantity": 1}], "order_price": -770.16}
6 }
7
8 {
9 "order_id": "85a2f99e-9c24-4c06-bd1f-d2a17b17abdc",
10 "error_type": "ORDER_MODE_CONFLICT",
11 "error_reason": ["ORDER_MODE_CONFLICT"],
12 "original_order": {"order_id": "85a2f99e-9c24-4c06-bd1f-d2a17b17abdc",
"user_id": "user_000087", "restaurant_id": 3086, "restaurant_name":
"Pind Balluchi", "restaurant_city": "Brigade Road", "cuisines": ["
North Indian", "Mughlai"], "phone_number": "08041153761", "
request_online": true, "request_table": true, "order_time":
"2026-05-23T07:58:21.725519Z", "items": [{"food_item": "Noodles", "
category": "Mughlai", "unit_price": 168.31, "quantity": 1}, {"
food_item": "Biryani", "category": "North Indian", "unit_price":
184.24, "quantity": 1}, {"food_item": "Dal Makhani", "category": "
Mughlai", "unit_price": 253.65, "quantity": 1}], "order_price":
606.2}
13 }
14
15 {
16 "order_id": "d5602ee8-d3e7-44b3-a999-305c402bb11f",
17 "error_type": "INVALID_PHONE",
18 "error_reason": ["INVALID_PHONE"],
19 "original_order": {"order_id": "d5602ee8-d3e7-44b3-a999-305c402bb11f",
"user_id": "user_000085", "restaurant_id": 8534, "restaurant_name":
"Swing", "restaurant_city": "Indiranagar", "cuisines": ["North
Indian", "Chinese", "Mangalorean"], "phone_number": "09190631936", "
request_online": false, "request_table": false, "order_time":
"2026-05-23T07:26:05.388827Z", "items": [{"food_item": "Unlisted
Mystery Dish", "category": "Unknown", "unit_price": 289.23, "
quantity": 1}, {"food_item": "Fish Thali", "category": "Mangalorean
", "unit_price": 230.12, "quantity": 1}], "order_price": 519.35}
20 }

```

Listing 3: Sample Output - Error Log Schema

5 Conclusion

The implementation of the stateless Kafka consumer successfully satisfies the requirements for Q2 of the Ashpaz pipeline. By segregating validation logic from persistent storage, the ingestion layer maintains extremely low latency and guarantees that downstream analytical jobs receive exclusively sanitized and structurally sound data.